Accuracy summary: all results

Every number here is copied from the result files in `experiments/`. All test sets were locked (frozen) before the runs that are reported on them.

There is no single "accuracy" for this project, because it has two modes that do different jobs. Use the number that matches the mode.

1. The short answer

Mode	Measure	Result
One function, fine-tuned model, single agent (arm C)	Refactoring accepted (all checks pass)	**79.5%** (70/88)
One function, fine-tuned model, multi-agent (arm D)	Refactoring accepted (all checks pass)	**39.8%** (35/88)
Whole project, final version, 14B model	Known problems found	**58.5%** (31/53)
Whole project, final version, 14B model	Logic bugs found	**62.5%** (15/24)
Whole project, final version, 14B model	Security problems found	**75.0%** (6/8)
Whole project, static checks only (no AI)	Known problems found	**34.0%** (18/53)
Whole project, all runs	Fixes marked "ready" that broke a test	**0 of 11**

2. Mode 1: one function (guard-clause refactoring)

Protocol `eval-v1`, test split of 88 unseen cases, run on Kaggle. Every rate counts failed runs against the arm.

Measure	A: base model, 1 agent	B: base model, multi-agent	C: fine-tuned, 1 agent	D: fine-tuned, multi-agent
Accepted (primary measure)	7/88 (8.0%)	1/88 (1.1%)	**70/88 (79.5%)**	35/88 (39.8%)
Output is valid Python	87/88 (98.9%)	88/88 (100%)	88/88 (100%)	88/88 (100%)
Follows the guard-clause rules	13/88 (14.8%)	9/88 (10.2%)	88/88 (100%)	88/88 (100%)
Behavior kept (reference tests pass)	68/88 (77.3%)	69/88 (78.4%)	70/88 (79.5%)	69/88 (78.4%)
Rejected	80/88 (90.9%)	85/88 (96.6%)	18/88 (20.5%)	19/88 (21.6%)
Uncertain	0/88	2/88 (2.3%)	0/88	34/88 (38.6%)
Median time per case	3.0 s	16.0 s	7.7 s	17.9 s

Paired tests on acceptance (exact McNemar, Holm adjusted):

Comparison	What it isolates	Only first accepted	Only second accepted	p
C vs A	Fine-tuning, 1 agent	65	2	< 0.0001
D vs B	Fine-tuning, multi-agent	34	0	< 0.0001
D vs C	Adding the other agents	1	36	< 0.0001
B vs A	Adding the other agents, base model	0	6	0.031

What this means:

- **Fine-tuning is what works.** It moved acceptance from 8.0% to 79.5%.
- **The multi-agent version accepts less than the single agent** (39.8% against 79.5%), and the reason is a technical failure, not the review's judgment. In all 34 uncertain cases of arm D the code passed syntax, the guard-clause rules and the reference tests, but the review agent's answer was cut off and could not be read. Those cases never received a verdict, so arm D's real ability with a working review is unmeasured. The protocol was frozen, so this was not fixed and rerun.
- Arm D missed the target set before the run: 39.8% against a floor of 43.5%. Arms A, B and C met theirs.
- Validation split, for comparison (84 cases): A 7.1%, B 3.6%, C 82.1%, D 54.8%.
- 84 of the 88 test cases come from the team's templates, 3 from MBPP and 1 from HumanEval, so the result says little about other sources.

Fine-tuning details: LoRA adapter on Qwen2.5-Coder-0.5B, 627 training records, 84 validation, 88 test. Best validation loss 0.03596 at epoch 3. This loss is not an accuracy figure; use the table above.

3. Mode 2: whole project review

Each test set is 8 small Python projects with problems written in on purpose, each with hidden tests. "Found" means a separate judge, who did not know which system produced the finding, confirmed that the finding describes the known problem. A finding that only lands on the right line does not count.

Final version (test set 3, never used for tuning)

Prompt version `project-v5`, Qwen2.5-Coder-14B, run under a full freeze. All 784 model calls succeeded.

Problem type	Static checks only	Full system
All problems	18/53 (34.0%)	**31/53 (58.5%, 95% CI 45% to 71%)**
Logic bugs	5/24 (20.8%)	**15/24 (62.5%)**
Security	3/8 (37.5%)	**6/8 (75.0%)**
Architecture	4/8 (50.0%)	4/8 (50.0%)
Code quality	6/13 (46.2%)	6/13 (46.2%)

The full system found 13 problems the static checks missed and missed none that they found (exact McNemar p = 0.00024).

How the result improved

Each row is a separate frozen run. Static checks alone are shown for the same set.

Set	Pipeline and model	Static only	Full system
Test 1	`project-v3`, 3B (fits the laptop)	18/55 (32.7%)	19/55 (34.5%)
Test 1	`project-v3`, 14B	18/55 (32.7%)	27/55 (49.1%)
Test 1	`project-v3`, 32B	18/55 (32.7%)	27/55 (49.1%)
Test 2	`project-v4`, 14B	19/55 (34.5%)	28/55 (50.9%)
Test 3	`project-v5`, 14B (**final**)	18/53 (34.0%)	**31/53 (58.5%)**

The 14B and 32B rows on test 1 were run after the 3B result was known. They changed only the model.

Fixes (final version)

Measure	Result
Fixable bugs that some proposed fix solved	16/32 (50.0%)
Fixable bugs solved by fixes the tool marked "ready"	3/32 (9.4%)
Fixes marked "ready"	4 (3 solve their bug, 1 has no effect on the hidden tests)
Fixes marked "ready" that broke a test	0
Proposed fixes that would break a test	33 (32 rejected by the checks, 1 left as uncertain)

Read these two rows together. The 50% is what the fixes can do when each is tried against the hidden tests. Most of them are not marked "ready", so a user would have to read them and accept them by hand. What the tool solves on its own confidence is 9.4%.

Across all five full-system runs, 11 fixes were marked "ready", none broke a test, and 110 harmful proposals were stopped before reaching "ready".

False alarms (final version)

Measure	Static only	Full system
Total findings	25	174
Findings on a known problem	24 (96%)	69 (40%)
Findings on correct code planted as a trap	1	10
Trap locations flagged	1/16 (6%)	8/16 (50%)
Findings elsewhere (not judged)	0	95

The system finds more but is noisy. The 95 "elsewhere" findings were not judged, so they may be false alarms or real problems nobody planted. True precision is therefore unknown.

Reference: Claude on the same projects (test set 2 only)

Problem type	Static only	Our system (14B)	Claude
All problems	19/55 (34.5%)	28/55 (50.9%)	49/55 (89.1%)
Logic bugs	5/24	13/24	24/24
Security	3/8	4/8	8/8
Architecture	4/8	4/8	8/8
Code quality	7/15	7/15	9/15

This is a ceiling for orientation, not a fair contest. The projects and the judge are also Claude models, which favors Claude. Claude proposed no fixes here, so the fix safety numbers do not apply to it. This comparison was not run on test set 3.

4. Limits to state when using these numbers

1. **The best project numbers need the 14B model, which does not fit the demo laptop** (4 GB GPU). They were run on Kaggle (2 x T4). The laptop can run 3B, which found 34.5% on test set 1. The final version `project-v5` was not measured with 3B.

2. **The test projects are small and synthetic** (8 projects per set, 449 to 686 lines of package code each), and they were written by language models. These numbers do not predict performance on large real codebases.
3. **The judge is a language model**, and the labels are in the repository so they can be checked.
4. **The sets are small**, so the intervals are wide: 58.5% has a 95% interval of 45% to 71%.
5. **Before test sets 2 and 3 were frozen**, an automatic security review showed the developer some of their seeded problems (11 and about 17). No review code or prompt was changed because of that; details are in `docs/evaluation/project-benchmark.md`.
6. A review takes about 17 minutes per project on Kaggle with the final version (median 1,047 s), against about 2 seconds for static checks alone.

5. Where the numbers come from

Result	File
One function, test split	`experiments/evaluation/test-2026-10-21/report/kaggle-grad-refactor-ai-eval-3ba44332/summary.md`
One function, targets	`experiments/evaluation/test-2026-10-21/targets-report/targets.csv`
One function, validation	`experiments/evaluation/validation-2026-10-21/report/`
Fine-tuning run	`experiments/training/full-2026-10-21.json`
Project review, test 1 (3B, 14B, 32B)	`experiments/evaluation/project-bench/report-models.md`
Project review, test 2 and Claude	`experiments/evaluation/project-bench/test2/report-with-claude.md`
Project review, final (test 3)	`experiments/evaluation/project-bench/test3/report.md`
Scoring rules and freeze procedure	`docs/evaluation/project-benchmark.md`